Chapter 3

JavaScript Foundations in the AI Era

Table of Contents

Chapter 3 1

JavaScript Foundations in the AI Era 1

3.1. Introduction JavaScript, variables, and data type 3

3.1.1. Basic JavaScript Instructions 6

3.1.2. Variables 8

3.1.3. Data Types 10

3.1.4. The prompt() method 12

3.2. Strings 15

3.2.1. Strings are indexed 15

3.2.2. Strings methods and properties 16

3.3. Arrays 21

3.3.1. Accessing array elements 22

3.3.2. String as array 23

3.3.3. String size 24

3.3.4. Properties and methods in arrays 24

3.4. conditional and logic statement 26

3.4.1. true or false values 26

3.4.2. conditional if statement 28

3.4.3. isNaN() function 32

3.4.4. Nesting conditions 33

3.4.5. True and false values 34

3.4.6. Logical operators 35

3.4.7. Switch 38

3.5. loops 39

3.5.1. for loop 40
3.5.2. while loop 44
3.6. Functions 48
3.6.1 Creating a function: user defined functions 49
3.6.2. Built-in function and methods. Math object 54
3.6.3. Global and Local Scoping in JavaScript 56
3.6.4. Save function in a variable 60
3.6.5. Higher order functions 62
3.6.6. Spread 64
3.7. Try and catch 66
3.8. JavaScript Objects 70
3.8.1. object definitions 70
3.8.2. What is "this"? 71
3.8.4. Object constructor using functions. 72
3.8.5. Methods 73

3.1. Introduction JavaScript, variables, and data type

JavaScript is a high-level, scripting language, interpreted programming language. Alongside HTML, CSS, JavaScript is one of the three core technologies of web developing. It is used to make webpages interactive. The filename extension for JavaScript is **.js**



JavaScript is a scripting programming language, which is a program that is written for a special run-time, step-by-step, environment. In JavaScript, whatever happens first happens first and then the next line happens afterward.

The power of programming language is the ability to perform the same operation on many different objects, or change that object itself over and over.

How HTML, CSS, and JavaScript fit together?

We developers usually talk about three languages that are used to create web pages: HTML, CSS, and JavaScript. Where possible, aim to keep the three languages in separate files, with the HTML page linking to CSS and JavaScript files.

html files

This is where the content of the page lives. The HTML gives the page structure and adds semantics.

css files

The css enhances the html page with rules that state how the html content is presented (backgrounds, borders, box, etc)

js files

This is where we can change how the page behaves, adding interactivity.

Example 1) comparison of HTML with CSS and then with CSS and JavaScript

HTML code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Introduction to JS</title>
</head>
<body>
    <div class="qcc">
        <h1>Queensborough Community College</h1>
        <div class="et">
            Department of Engineering Technology
        </div>
    </div>

</body>
</html>
```

The internet browser of the HTML file will look as follows:



Adding the CSS to the HTML file:

CSS file

```
body{
    background-color: lightblue;
}
div.qcc{
    background-color: darkblue;
    text-align: center;
    color: aliceblue;
```

```
    padding-top: 20px;
    height: 7rem;
}
div.et{
    border: ridge 2px white;
    padding: 10px;
    background-color: crimson;
    width: 80%;
    margin: 0px auto;
    font-size: 1.5rem;
    font-style: italic;
}
```

The internet browser will look as follows:



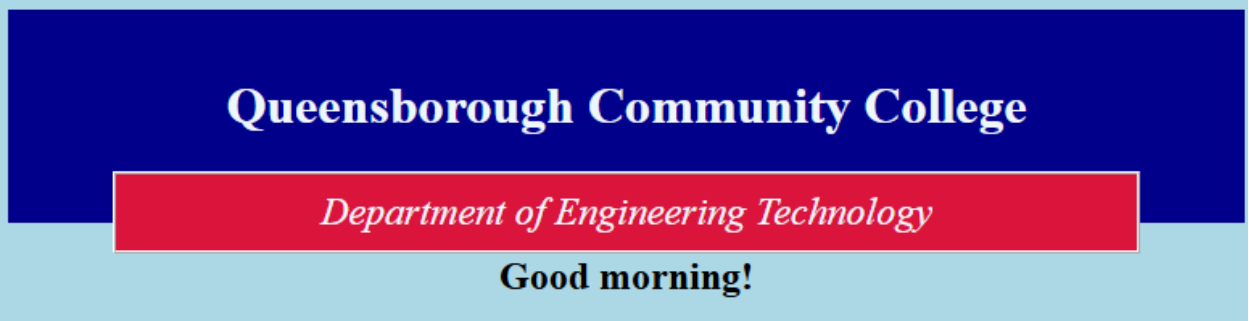
Lastly, if we link the following JS file to the HTML file:

JS file

```
let today=new Date();
let hourNow = today.getHours();
let greeting;

if(hourNow >24){greeting = 'Welcome to the next day'}
else if(hourNow>18){greeting = 'Good evening!'}
else if(hourNow>12) {greeting = 'Good afternoon!'}
else if (hourNow>0) {greeting = 'Good morning!'}
else{greeting = 'Welcome!'}
document.write('<h2 class="greet">' + greeting + '</h2>')
```

If the internet browser opens before 12 PM, for example 10:11 AM, it will look as follows:



If the internet browser opens after 12 PM, for example 2:43 PM, it will look as follows:



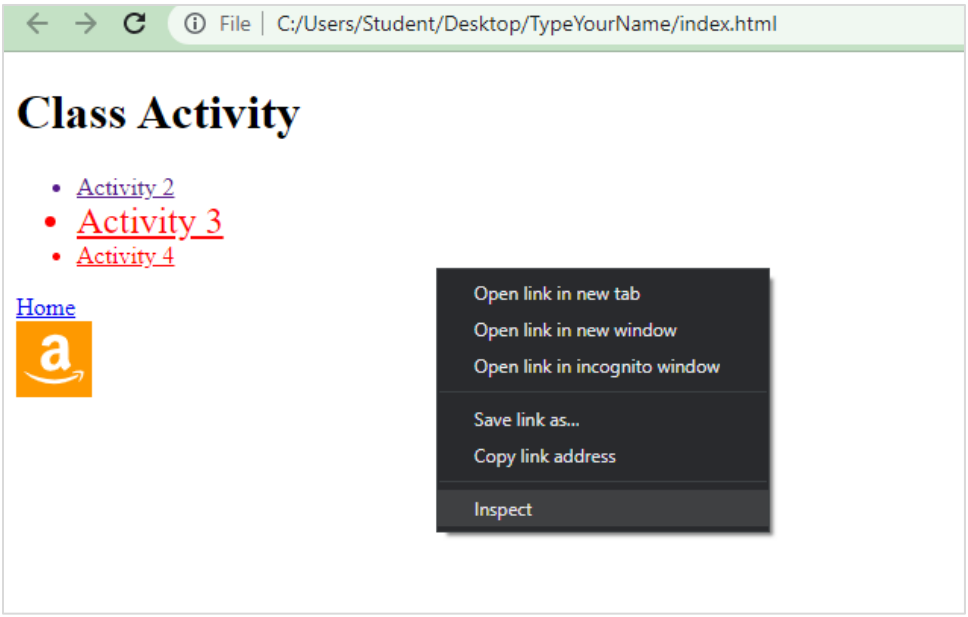
3.1.1. Basic JavaScript Instructions

Writing and debugging

The JavaScript console is a very useful tool for learning JavaScript fundamentals and debugging your code. It allows you to test code snippets, inspect errors, and better understand how your programs work.

In Google Chrome, you can open the JavaScript Console by navigating to **View** → **Developer** → **JavaScript Console**. On Mac, press **Cmd + Option + I**. On Windows, press **Ctrl + Shift + I** or simply use the **F12** key. An alternative way is to right-click anywhere on the webpage and select '**Inspect**', which is the last item in the menu.

To test JavaScript code, you can create an HTML file and link it to a JavaScript file. After opening the HTML file in your browser, open the Developer Console (F12) to view and test your JavaScript code.



Once the developer tools is open, on the top right of the tools bar, we can see the Console tab.

We can also use the console to have a quick check, to troubleshoot, or to test a JS code, but never to write a complete JS code.

A script is a series of instructions that a computer can follow one-by-one. Each individual instruction or step is known as **statement.** Statements should end with a semicolon **;**

JavaScript is case sensitive so a variable named **hourNow** is not the same as variable **HOURNOW** or **HourNow**.

For example, the following .js file

```
/*This script will display the greeting message according to the
system time obtain using the command today.getHours() */

let today=new Date();
let hourNow = today.getHours(); var greeting;

if (hourNow > 18)
     { greeting = 'Good evening!'; }
else if (hourNow > 12)
     { greeting = 'Good afternoon!'; }
else if (hourNow > 0)
```

```
     { greeting = 'Good morning!'; }
else
     { greeting = 'Welcome!'; }

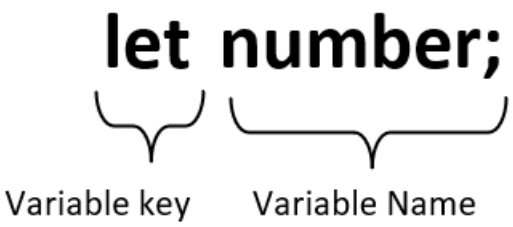
document.write('<h2>' + greeting + '<h2>')
```

- Each of the line of code between /* and */ are multi-line comments.
- The lines with the let word and if, else if, else are statements.
- The curly braces indicates that start and end of a code block. Each code block could contain many more statements.
- The code if, else if, else is a decision statement that indicates which code should run.

You can also explore the demo sample at CodePen Demo to see how modifying JavaScript instructions can change the behavior and interaction of HTML components.

3.1.2. Variables

Variables in JavaScript are containers for temporary storing data values. JavaScript traditionally uses the keyword **var** to declare a variable. But in the latest years, JS came out with a new method to declare variables known as **let:**



Rules for naming variables

When naming JavaScript variables, developers should use clear and descriptive names that follow camelCase conventions and avoid reserved JavaScript keywords. The basic rules for naming variables are:

1. The name must begin with a letter, an underscore_, or money sign $
   ```
   let x1;
   let _value;
   ```

```
let $money;
```

2. Variable names CAN'T BE keyword or reserved words. For example, let var = 3;
   - **var** is not a valid name for a variable because **var** is a keyword

   Some of the JavaScript reserved keywords are:

JavaScript Reserved Keywords				
break	default	function	return	var
case	delete	if	switch	void
catch	do	in	this	while
const	else	instanceof	throw	with
continue	finally	let	try	debugger
for	new	typeof		

Table 1 – JavaScript reserved keywords

3. Variable names CAN'T start with a number, any other symbols or characters, and no space as the following variable declarations:

```
let 8value;

let $99;

let white-car;

let first name;

let switch;
```

4. All variables are case sensitive, so the variable **color** is different than variable **Color** or **COLOR**, but it is bad practice to create two variables that have the same name using different cases.

```
let color = 'beige';

let Color = 'red';
```

```
let COLOR = 'blue';
```

5. Use name that describes the kind of information that the variable stores. For example, **firstName** might be used to store a person's first name, and **id** for their id number. Note: try to avoid long words or names. For example, instead of naming a variable **programming1**, you can name it **prog1**

3.1.3. Data Types

JavaScript data types define the kind of values a variable can store, such as strings, numbers, booleans, objects, and arrays.

Numeric data

Numeric data handles numbers. To store a numeric data type in a variable, simply assign a number to it, could be positive, negative, or fraction.

```
let x = 100; let y = -100;
```

String data

The string data type handles strings or letters and symbols, what we commonly refer to as words and sentences. To store a string data type in a variable, wrap your string of text in straight quotation marks. You can use single or double quotes. Just make sure you use the same one at the beginning and at the end.

```
let color1 = 'beige';
let string1 = "Strings are typically words and sentences";
let string2 = 'I want to keep the "quotation" mark';
let string3 = "I want to keep the \"quotation\"";
```

Boolean data

The Boolean data type handles the binary true/false. To store a Boolean data type in a variable, type in true or false without quotation marks. The Boolean data type handles the binary true/false. To store a Boolean data type in a variable, type in true or false without quotation marks.

```
let boolean1 = true; let boolean2 = false;
```

The null data

The null data type is the intentional absence of an object value. So if we want a variable to be empty, but not undefined, you set its value to null, without quotation marks as well.

```
let emptyVar = null;
```

Undefined data

The undefined data type is what you get when you create a variable but do not set it to anything. As you've learned, JavaScript is a weakly typed language, so the data types are applied when you set the content in a variable.

```
let undefined1;
```

How do you check the data type of value in a variable?

In JavaScript, to check the type of value in a variable, we can use the **typeof** operator. The **typeof** operator returns a string indicating the type of the unevaluated operand.

```
let nothing= null;
typeof(nothing)          // result should be 'object'
let emptyVar="";
typeof(emptyVar)         // result should be 'string'
```

Template literals

Using b**ackt**icks `` ` `` in JavaScript along with **${}** around the expressions allows to embed information in the **${}** into the string.

Concatenating strings

```
let a = 5;
let b = 10;
console.log('Fifteen is ' + (a + b) + ' and\nnot ' + (2 * a + b)
+ '.');
```

Using backticks

```
let a = 5;
let b = 10;
console.log(`Fifteen is ${a + b} and not ${2 * a + b}.`);
```

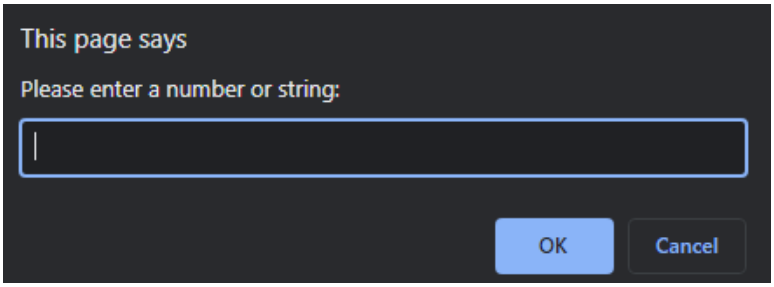
3.1.4. The prompt() method

The **`prompt()`** method displays a dialog box that prompts the user for input. The **`prompt()`** method returns the input value if the user clicks "OK", otherwise it returns null.

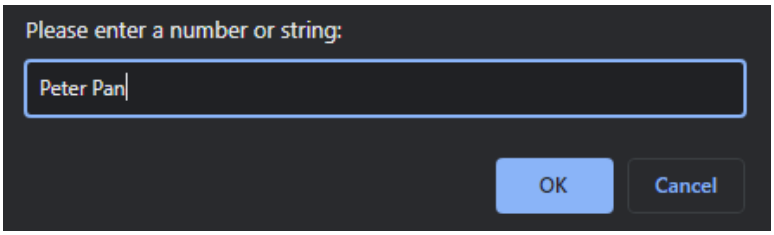
Example 2) use **`prompt()`** method to collect a name from a keyboard. Then display the name in the console.

```
let name= prompt('Please enter a name: ');
console.log(`The entered name was: ${name}`);
```

When the program runs, a dialog box appears as follows:



If the user type Peter Pan and click OK



The value entered in the dialog box will be displayed in the Console as: "The `entered name was: Peter Pan.`"

Conversion Between Data Type

Convert from string to integer

The value collected from the **`prompt()`** dialog box is a string. If we want to convert the value into an integer, we need use method **`parseInt()`** to convert

the string into an integer.

Example 3) collect two numbers from the **`prompt()`** dialog box, sum the two numbers and print the result in the console.

Data collected from a dialog box is treated as a string. If two numbers are entered and the + operator is used, JavaScript will concatenate them instead of performing an arithmetic addition.

For the following code:

```
let number1 = prompt("Enter a number: ");
let number2 = prompt("Enter a second number: ");
let sum = number1 + number2;
console.log(`${number1} + ${number2} = ${sum}`);
```

When the program runs, a dialog box appears. If the number 2 is entered as the first input and 5 as the second, the console will display:

```
2 + 5 = 25
```

Now, if we convert or cast **number1** and **number2** to integers first before adding them:

```
let number1 = prompt("Enter a number: ");
// Convert the first input from a string to an integer
number1 = parseInt(number1);
// Prompt the user to enter the second number and
immediately convert it to an integer
let number2 = parseInt(prompt("Enter a second number: "));

let sum = number1 + number2;
console.log(`${number1} + ${number2} = ${sum}`);
```

When the program runs, a dialog box appears. If the number 2 is entered as the first input and 5 as the second, the console will display:

```
2 + 5 = 7
```

Exercise 1) Personal Profile Generator (AI-Assisted)

Create a small JavaScript program that collects user input using **prompt()**, correctly handles data types, and displays a formatted message using template literals.

Instructions

Write a JavaScript program that:

1. Ask the user for their:
    - First name (string)
    - Age (number)
    - Favorite programming language (string)
2. Convert the age input into a **number data type**.
3. Stores all values in variables with appropriate names.
4. Uses **template literals** to display a message in the console.

Expected Output Example

If the user enters:

- `Name: Peter`
- `Age: 25`
- `Language: JavaScript`

The console should display:

```
My name is Peter. I am 25 years old and my favorite
programming language is JavaScript.
```

AI Assistance Challenge

Before writing your final answer, use AI to:

- Check if your variables are correctly typed (string vs number)
- Verify your template literal syntax

- Improve your sentence structure for readability

Bonus Challenge (Optional)

Modify your program to also ask:

- Favorite hobby

Then extend your output like this:

```
My name is Peter. I am 25 years old, I love JavaScript, and
my favorite hobby is football.
```

3.2. Strings

Strings in JavaScript are objects that represent a sequence of characters. They are used to store and manipulate text.

A JS string could have zero or more characters. Those characters are written inside, between, a single or double quotes.

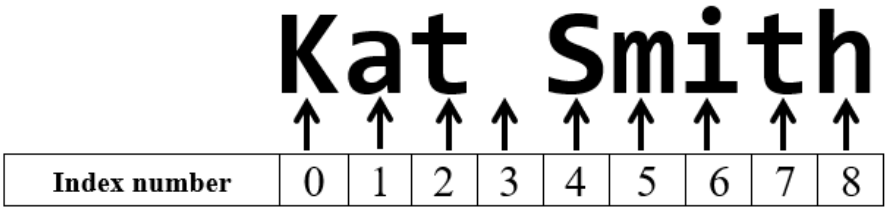
```
let name1 = "Peter Smith";
let name2 = 'Kat Smith';
```

You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
let answer1 = "It's alright";
let answer3 = 'He is called "Johnny"';
```

3.2.1. Strings are indexed

Strings are considered an array of characters, therefore, each character in a string has index. The first character in a string, the leftmost character, has an index of zero, and the index number increases as we count from left to right of the string



To access to a character in a string, we simply type the string name and type the index number in between a squared brackets right next to the string name:

```
let name = 'Kat Smith';
name[2]   // t
name[30]  // undefined
```

3.2.2. Strings methods and properties

Methods are function belonging to an object. The context of a method (this value) equals the object the method belongs to. Methods are functions stored as object properties. Methods can be accessed with the following syntax:

```
Object.methodName();
```

On the other hand, methods used to obtain information about an object are known as accessor methods or properties. Properties can be accessed as:

```
Object.propertyName;
```

Checking the length of a string

One property that you can use with strings is the **.length**, which returns the number of characters contained in the string object.

```
let text = "Hello World!";
let msgLength = text.length;
console.log(msgLength);  // 12
```

Converting to Upper and Lower Case

A string can be converted to upper case with **`toUpperCase()`** or to lower case with **`toLowerCase()`** method.

```
let myString = "Queensborough Community College";
console.log(myString);
let myUpperString = myString.toUpperCase();
console.log(myUpperString);
```

The console should print:

```
Queensborough Community College
QUEENSBOROUGH COMMUNITY COLLEGE
```

Searching for a character in a string

JS uses the method **`search()`** to find characters within a string. It returns the location of the first searched character. The characters that we are looking for are placed in between the parenthesis. Also, for the count of the character position, the first character position has an index of zero.

```
let myString = "Queensborough Community College";
console.log(myString);
let stringLocation = myString.search('Co')
console.log(stringLocation);
```

The console should print:

```
Queensborough Community College
14
```

Remove white spaces

Method **`trim()`** removes the white spaces at the beginning and end of a string but not the space in between word.

```
let name ="       Willim Smith";
name.trim();
```

The console should print without the left whitespace:

```
Willim Smith
```

Combining methods

We can also combine methods by using a dot in between methods.

```
let name = "        Hello           World            ";
name.toUpperCase().trim();
```

The console should print without the left whitespace:

```
Hello           World
```

Method with argument

Subtracting characters

The method uses to subtract characters from a string is **substr(x,y).** The **substr(x,y)** method has two parameters, x = the position of the character, within the string, and y = the number of character subtracted from the position.

Syntax

```
substr(position of character, number of character to be
subtracted from the position)
```

```
let myString = "Queensborough Community College";
let subWord = myString.substr(6,5);
console.log(subWord);
```

The console should print without the left whitespace:

borou

```
let myString = "Queensborough Community College";
console.log(myString);
let stringLocation = myString.search('Co')
let substrLocation = myString.substr(stringLocation,5);
console.log(substrLocation);
```

The console should print without the left whitespace:

```
Queensborough Community College
```

Commu

Replacing string content

The **replace(x,y)** method replaces a specified value with another value in a string. The method has two attributes, x = the character/s to be replaced and y = the replace characters.

```
let myString = "Queensborough Community College";
console.log(myString);
let replaceString = myString.replace('Community','PARK')
console.log(replaceString);
```

The console should print without the left whitespace:

```
Queensborough Community College
Queensborough PARK College
```

Another example could be:

```
let myString = "Queensborough Community College";
console.log(myString);
let replaceString = myString.replace('Co','PARK')
console.log(replaceString);
```

The console should print without the left whitespace:

```
Queensborough Community College
Queensborough PARKmmunity College
```

Extracting characters from a string

The **`slice()`** method extracts a section of a string and returns it as a new string without modifying the original string.

```
const str = 'The quick brown fox jumps over the lazy dog.';
newString = str.slice(31);
console.log(newString)
```

The console should print:

```
the lazy dog.
```

If we slice from index 4 through 19:

```
newString = str.slice(4,19);
console.log(newString)
```

The console should print:

```
quick brown fox
```

If we slice from index -6:

```
newString = str.slice(-6);
console.log(newString)
```

The console should print:

```
y dog.
```

If we slice from index -12 and -5:

```
newString = str.slice(-12, -5);
```

```
console.log(newString)
```

The console should print:

```
he lazy
```

If we slice from index -5 and -12:

```
newString = str.slice(-5, -12);
console.log(newString)
```

The console should print:

```
''
```

3.3. Arrays

An array is literally an array where you can store as few or as many items as you like and retrieve one of, some of, or all of those items.

There are different ways to create an array. Manly, to create an array using an array initializer, first create an undefined variable, then add the different values to the array using a comma separated list wrapped in square brackets.

```
let fruits;
fruits = ['oranges', 'apples', 'peaches']; //Create an array
using square brackets
```

Also, instead of a two lines code, you can write the code in one line of code

```
let fruits=['oranges','apples','peaches']; //Create an array
using square brackets
```

Arrays can also be created by using a constructor **`new Array`** with an single number parameter

```
fruits = new Array ('oranges', 'apples', 'peaches');
```

One interesting note about arrays each value in the array can be a separate data type so you can mix strings, Booleans, and numbers all you like.

```
let mixedArray =['cars', 10, true,
'mixed array'];
```

3.3.1. Accessing array elements

Each of the values added to an array is placed in a separate slot and given an index number. **Computer indexes start with zero** so the first item has index number zero, the second number is indexed one, and so on. Once the array is created, you can retrieve a specific value from the array based on its index.

```
fruits = new Array ('oranges', 'apples', 'peaches');
let x = fruits[2];  // Store string “peaches” in variable x
```

If you need to change a value in the array you simply refer to its index position and assign a new value.

```
fruits = new Array ('oranges', 'apples', 'peaches');
// replace the string 'oranges' with 'cherries'
fruits[0] = 'cherries';
```

Subarrays

You can also create arrays inside of an array. To do so, we just need to open a square brackets inside of a square brackets:

Syntax

```
let meals= ['breakfast',['egg','bread','juice'],
'lunch',['fish','salad']];
console.log(meals)
```

The console should print:

```
(4)[ 'breakfast', Array(3), 'lunch', Array(2)]
```

Another example of subarray is:

```
let countries = ['America',['Canada', 'USA', 'Mexico', 'Brasil',
'Argentina'], 'Europe',['France', 'England', 'Italy', 'Spain',
'Germany','Switzeland'], 'Asia',['China', 'India', 'Japan',
'Korean', 'Singapore'] ];
```

The console should print:

```
contries
(6)[ 'America', Array(5), 'Europe', Array(6), 'Asia', Array(5)]
```

To retrieve information from a subarray, we write square brackets of the array position follow by the square bracket of the subarray index.

```
console.log(countries[1][3]);
```

The console should print:

```
Brasil
```

3.3.2. String as array

A string can also be treated as an array. We create an string and retrieve the character by using square brackets and the index position of the character.

```
let country = "United States of America "
console.log(country[7])
```

The console should print:

```
'S'
```

3.3.3. String size

Also, we can use string properties such as l**ength** to retrieve elements out of an array

```
let country = "United States of America "
console.log(country.length)              // 24
console.log(country[country.length-1])   // a
```

3.3.4. Properties and methods in arrays

In JavaScript, objects have properties and methods. Properties are pieces of meta information about the object we can retrieve and use. Methods are functions that belong to that object. For more information about the different arrays and how to use then, go to w3schools.com or developer.mozilla.org

Direct methods

These are the methods that update the array information directly within the array:

```
let mixedArray =['cars', 10, true, 'apples'];
console.log("Before :", mixedArray);
//
// Reverse method mixedArray.reverse();
console.log("After reverse :", mixedArray);
//
// Remove the first value of the mixedArray using method "shift"
mixedArray.shift();
console.log("After shift method :", mixedArray);
//
// Add comma-separated list of values to the front of the array
using method "unshift"
mixedArray.unshift("NY", 280);
console.log("After unshift method :", mixedArray);
//
//add comma-separated list values at the end of the array using
method “push"
mixedArray.push(350, "QCC");
console.log("After push method :", mixedArray);
```

The console should print:



Methods that return values

slice method creates a copy of the array and then returns it to us, and typically we would place it inside a variable, or use it directly in some sort of function.

```
let fruits = ["apples", "cherries", "beaches"];
// create a copy of array fruits let fruits1 =ruits.slice();
console.log("Array fruits: ", fruits);
console.log("New fruits array using method slide : ", fruits1);
```

The console should print:

```
Array fruits: ["apples", "cherries", "beaches"]
New fruits array using method slice: ["apples", "cherries",
"beaches"]
```

The next method, **indexOf**, gives us the index number for a specific search. If the search is false, it returns a -1.

Syntax:

indexOf (value we are searching, from which index position);

```
let fruits1 = fruits.indexOf("cherries", 0);
console.log("Search of location of the cherries in array fruits
: ", fruits1);
console.log("To display value of searched index is: ",
fruits[fruits1]);
```

The console should print:

```
Search of location of the cherries in array fruits : 1
To display value of searched index is: cherries
```

join method takes all the elements in an array and join them together in a single string and separate them with a comma.

```
let fruits = ["apples", "cherries", "beaches"];
let fruits1 = fruits.join(" % ");
console.log(fruits1);
```

The console should print:

```
apples % cherries % beaches
```

Now, notice that the commas have no space after them, so if you want to use these as actual HTML, you probably want to change that separator. You can do that by changing it in the arguments between the parentheses.

3.4. conditional and logic statement

A conditional statement in JavaScript allows your program to make decisions by executing different code based on whether a condition is true or false.

3.4.1. true or false values

In JavaScript, the Boolean data type represents one of two values: true or false. Those values can also be assigned to a variable as:

```
let day = true;
let night = false;
```

The Boolean variables are later used in the program to test or compare if a given condition is true or false.

```
if (day){
     console.log("The statement is =  ", day);
}
```

The console should print:

```
The statement is = true
```

A lot of programming is about testing conditions, asking if some condition is met, then do this, otherwise do something else. This is done using conditional statements and logic operators.

- The **>** comparison operator checks if one value is greater than another value.
- The **<** comparison operator checks if one value is less than another value.
- The **>=** comparison operator checks if one value is greater than or equal to another value.
- The **<=** comparison operator checks if one value is less than or equal to another value.
- The **==** comparison operator checks if two values are equal.
- The **===** comparison operator checks if two values are exactly equal in both value and data type.
- The **!=** comparison operator checks if two values are not equal.
- The **!==** comparison operator checks if two values are not exactly equal in value or data type.
- The **&&** Boolean operator returns true only if all conditions are true.
- The **||** Boolean operator returns true if at least one condition is true.
- The **!** Boolean operator returns true when a condition is false and returns false when a condition is true.

Character or string comparison with **greater than > or less than <** with compare the character Unicode value:

```
"peter" > "Peter"   // true
"p" > "P"           // true
"p" < "P"           // false
```

== double equal sign checks for equality of value, but not equality of type. It basically treats both values as the same type and then compares them. This can be very problematic.

```
20 == 20            // true
```

```
"20" == 20          // true
0 == false          // true
1 == true           // true
true == true        // true
null == undefined   // true
```

=== triple equal sign checks for equality of type and value

```
 20 === 20          // true
"20" === 20         // false
0 === false         // false
1 === true          // false
true === true       // true
null === undefined  // false
```

!= not equal to compares if the values, and not the type, that are not equal to each other

```
5 != 5              // false
"5" != 5            // false
```

!== exactly not equal to compares if the values and the types are not equal to each other

```
5 !== 5             // false
"5" !== 5           // false
```

For comparison, it is recommended to use triple equal **===** and exactly not equal to **!==**

3.4.2. conditional if statement

if statement tests if a condition is true, then do something. It is also known as an ON/OFF switch condition.

Syntax

```
if (testing condition) { do something }
```

Here, the parenthesis wraps the condition, or conditions, and the curly braces wrap what happens if the condition is met. This is known as the code block. You could technically write this all on one line, but that would be hard to read for the humans who will be looking at your code, so you can write the first curly brace on one line, then the code block inside in the next line.

```
if (testing condition)
{ do something }
```

Example 4) Use if statement to check a person's mood

```
let goodMood = true;
let gotSleep = false;

if (goodMood === gotSleep){
      console.log("Today is a good day!");
}


console.log("I am moody!");
```

The console should print:

```
I am moody!
```

If we change the conditional to triple equal sign, we are basically comparing if the two logic statements are true.

```
let goodMood = true;
let gotSleep = true;

if (goodMood === gotSleep){
      console.log("Today is a good day!");
}

console.log("I am moody!");
```

The console should print:

```
Today is a good day!
I am moody!
```

if – else condition

If you want to create an **if/else** statement, where you perform a separate specific action **if** the condition is not met as well, you add **else**, and a new set of curly braces after the **if** statement. If some condition is true, do something, **else**, if the condition it's not true, then check something **else**. If statements require conditions to work, so we need some logic operators.

Example 5) Check if two numbers are equal

```
let a = 5, b =4, equalNum;
if (a===b)
     equalNum = true;
else
     equalNum = false;

console.log("Are the number the same? = " + equalNum);
```

The console should print:

```
Are the number the same? = false
```

if – else if - else condition

`if-else if-else` condition tests more than two statements.

```
let num1 = 5, num2 = 8;
if(num1 ===num2)
     console.log("the numbers are equal")
else if (num1 > num2)
     console.log("number 1 is greater than number 2")
else if (num2 > num1)
     console.log("number 1 is less than number 2")
else
     console.log("invalid entry")
```

The console should print:

```
number 1 is less than number 2
```

Example 6) use an **if, else if** statement to check which color is selected from an array

```
let colors =['red','blue','green','yellow','orange']

let userPick = parseInt(prompt("Select a color from 1 to 5"))
// parseInt converts input from the prompt function into an integer
let selectColor = colors[userPick - 1]

if(selectColor === 'red')
     console.log('The selected color is red!')
else if (selectColor =='blue')
        console.log('The selected color is blue!')
else if (selectColor =='green')
        console.log('The selected color is green!')
else if (selectColor =='yellow')
        console.log('The selected color is yellow!')
else if (selectColor =='orange')
        console.log('The selected color is orange!')
else
        console.log('INVALID!')
```

The console should print:

```
The selected color is green
```

Remember that JavaScript allows data type collision, meaning that it can modify data to produce the same resulting data type, then it uses a triple equal sign **===** Boolean operator, to identify exact equal data values. For example, if we want to compare if number 5 and string value 5 are equal values, by using double equal signs, JavaScript will apply data type collision and will treat both values as the same value:

```
let num = 10;
let otherNum = "10";
if(num>otherNum)
      console.log("Number ", num, "is greater than ",otherNum)
else if(num==otherNum)
      console.log("Number ", num, "is equal to ",otherNum)
else if(num<otherNum)
      console.log("Number ", num, "is less than ",otherNum)
else
      console.log("Different data values");
```

The console should print:

```
Number 10 is equal to 10
```

Now, if we replace the double equal signs with the triple equal signs, then JavaScript will check for the exact the same type and value.

```
let num = 10;
let otherNum = "10";
if(num>otherNum)
      console.log("Number ", num, "is greater than ",otherNum)
else if(num===otherNum)
      console.log("Number ", num, "is equal to ",otherNum)
else if(num<otherNum)
      console.log("Number ", num, "is less than ",otherNum)
else
      console.log("Different data values");
```

The console should print:

```
Different data values
```

3.4.3. isNaN() function

isNaN(), is Not a Number, is a function that can check if a value is not a number. If the value is not a number, it returns true, else, if the value is a number, it returns false.

Example 6) write JS code that opens a dialog box and asks the user to enter a number. If the user enters a string, it will console a message as: _____ *is a string*. Otherwise, if the user enters a number, it will console a message as*: ____ is a number.*

```
let userInput = prompt("Enter a number: ")
let checkInput = isNaN(parseInt(userInput))

if(checkInput)
        console.log(`${userInput} is a string`)
else
        console.log(`${userInput} is a number`)
```

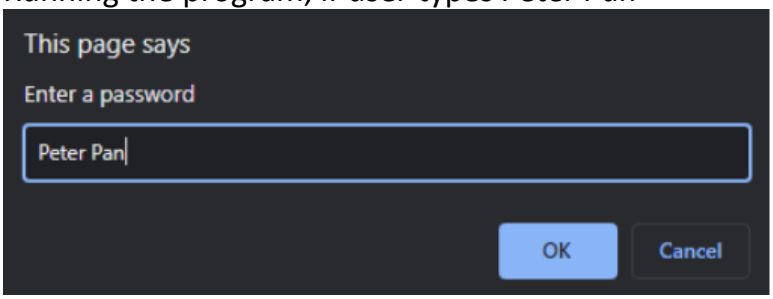
3.4.4. Nesting conditions

Nesting condition is when we have a condition inside of a condition. It basically checks another condition of a true condition.

Example 7) Create a JS code, using if-else statement, to check if a password is at least six characters not counting the space. If the password has more than six characters but it also has space, then the program will display an error message.

```
let password = prompt("enter a password: ")
if (password.length >= 6){
   console.log("password has 6+ characters! ")
   if (password.indexof(" ") === -1)
     console.log(`Great! ${password} has no space!`)
   else
     console.log("warning! password cannot have space ")
}
else
     console.log("password has less than 6 characters! ")
```

Running the program, if user types *Peter Pan*



and click OK, the console will print:

```
password has 6+ characters!
warning! password cannot have space
```

If the program runs again and the user types "Peter" and clicks **OK**, the console will display:




```
password has less than 6 characters!
```

If the program runs again and the user types “PeterPan” and clicks **OK**, the console will display:




```
password has 6+ characters!
Great! PeterPan has no space!
```

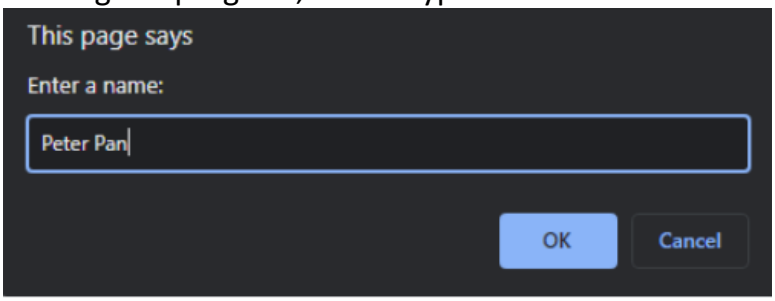
3.4.5. True and false values

Some statements or values are assigned to be true or false depending on the condition.

Example 8) Write a JS code that asks the user to enter a name. If the user enters a name, it will display a welcome message with the entered name. Otherwise, if the user does not enter a name, it will display that no name was entered.

```
let name = prompt("enter a name ")
if (name)
     console.log(`welcome to the program ${name}`)
else
     console.log(`you didn’t enter a name`)
```

Running the program, if user types *Peter Pan* and clicks **OK**, the console will display:




```
Welcome to the program Peter Pan
```

If the program runs again and the user doesn't type anything and clicks **OK**, the console will display:




```
You did not enter a name!
```

A null, empty, undefined, NaN, and 0 are inherent to be **false**. Otherwise, any string, numbers, or even a space are inherent to be true.

3.4.6. Logical operators

AND operator &&

In a chain of statements, compares if all statements are **true** for the entire statement to be **true**. In another words, the statement is true if all the conditions are true.

```
true && false && true       // false
true && true && true        // true
```

Example 9) Modify Example 7 so that instead of using a nested if-else statement, it uses an if-else statement with the AND (&&) operator.

```
let password = prompt("enter a password: ")
if (password.length >= 6 && password.indexof(" ") === -1){
     console.log(`Great! ${password} has no space!`)
else
     console.log("Error! password must have 6+ characters and no
space in between  ")
}
```

Example 10) Type a mystery word that will make the console display **"YOU GOT IT!!!"**. The user must guess a word that starts with the letter **"P"**, has a length greater than **5**, and contains the number **7** in the word.

```
const mystery = prompt("Enter a word that starts with P, has 5+
characters, and contains the number 7 in the word: ")

if(mystery[0] === "P" && mystery.length > 5 &&
mystery.indexOf("7") !== -1){
     console.log("YOU GOT IT!")
}
else{
     console.log("Wrong Guess!")
}
```

***OR operator* ||**

For the OR operator, we can state that from a chain of statements, if one statement is true, the entire statement is true as shown the following.

```
true || false || true        // true
true || true || true         // true
false || false || false      // false
```

Example 11) write a JS code that asks the user to rate a book as:

1. Excellent
2. Good/Average
3. Bored

Any other numbers will be considered an invalid choice

```
let rate = prompt("Enter a rate for The Book. Select: \n
1)Excellent \n 2) Good/Average \n 3) Bored")
if (rate === 3)
     console.log("We are sorry that you find The Book boring")
else if (rate == 2 || rate == 1)
     console.log("Thank you for choosing The Book")
else
     console.log("INVALID RATE!")
```

Combining logical operators

We can also combine logical operators in one statement. If you combine logical operators, AND operator has precedent over OR operator, meaning that a JS code always checks on the AND operator before the OR operator.

Example 12) Tickets to a park charge the following fee:

- Age 0 to 7, price FREE
- Age 8 to 17, price $5.00
- Age 18 to 64, price $8.00
- Age 65 and up, price FREE

Create a JS code that will display the price according to the age or status.

```
let age = int(prompt("enter an age: ")
if(age >= 0 && age <= 7 || age >=65)
     console.log("free tickte")
else if(age <= 17 && age > 7)
     console.log("$5.00 ticket per person")
else if(age < 65 && age > 17)
     console.log("$8.00 ticket per person")
else
     console.log("invalid age")
```

NOT ! operator

NOT operator returns a true if an expression is false.

```
!(7 === "7")    // true
!(9 === 6)      // true
!(8 === 8)      // false
```

3.4.7. Switch

Switch statement works the same as **`if, else if, else`** statement, with the difference that we select among different cases.

Example 13) use switch-case statement to select an animal from a list of animals

```
let selectedAnimal = prompt(
  "Choose an animal: lion, tiger, bear, zebra, or giraffe"
);

switch(selectedAnimal){
  case "lion":
    console.log("The selected animal is lion");
    break;

  case "tiger":
    console.log("The selected animal is tiger");
    break;

  case "bear":
    console.log("The selected animal is bear");
    break;

  case "zebra":
    console.log("The selected animal is zebra");
    break;

  case "giraffe":
    console.log("The selected animal is giraffe");
    break;

  default:
    console.log("Animal is not in the list!");
}
```

You can also group multiple cases together to produce the same output by placing one case after another as:

```
let selectedAnimal = prompt(
  "Choose an animal: lion, tiger, bear, zebra, or giraffe"
);

switch(selectedAnimal){
  case "lion": case "LION": case "Lion":
    console.log("The selected animal is lion");
    break;

  case "tiger": case "TIGER": case "Tiger":
    console.log("The selected animal is tiger");
    break;

  case "bear": case "BEAR": case "Bear":
    console.log("The selected animal is bear");
    break;

  case "zebra": case "ZEBRA": case "Zebra":
    console.log("The selected animal is zebra");
    break;

  case "giraffe": case "GIRAFFE": case "Giraffe":
    console.log("The selected animal is giraffe");
    break;

  default:
    console.log("Animal is not in the list!");
}
```

3.5. loops

In the real world, that is rarely how things work and, in many cases, we specifically want our code to do things more than once, usually as many times as is necessary and sometimes even endlessly. For this, we have loops. Loops are a vital part of all programming languages and will play a vital role in most JavaScript code. At their core,

loops are simple. We create some sort of loop condition and as long as this condition holds or **true**, the loop will keep running.

3.5.1. for loop

for loop runs the statement as long as the condition is true.

Syntax

```
for (initial_value; loop condition; update_initial_value) {
  // code block to be executed
}
```

Example 14) use a *for* loop to display an increasing counter from 0 to 4

```
for(let counter = 0; counter<=4; counter++){
  console.log(`The current counter is ${counter}`)
}
```

When the program runs, the following output will be displayed in the console:

```
The current counter is 0
The current counter is 1
The current counter is 2
The current counter is 3
The current counter is 4
```

Example 15) write a JS code that uses **for** loop to print even numbers between 0 and 100, exclusive. The initial number will be collected from the dialog box

```
let num = parseInt(prompt("Enter a number between 0 and 90"))
for(num; num<100; num++){
   if(num%2===0){
      console.log(num)
   }
}
```

When the program runs, if the user enters 90, the console will display all even numbers starting from 90 up to 98:

90

92

94
96
98

The for loop starts at the number entered by the user and continues while the number is less than 100. The if statement checks whether the number is even using num % 2 === 0.

Example 16) write a JS code that uses a *for* loop to print numbers from 100 to 0, inclusive, with a decrement of 10

```
for(let num = 100; num>=0; num -=10){
    console.log(num)
}
```

Infinite loops

Infinite loops are loops that do not stop; they run forever! Infinite loops are not very recommended since they can use all our computer memory and overheat our microprocessor.

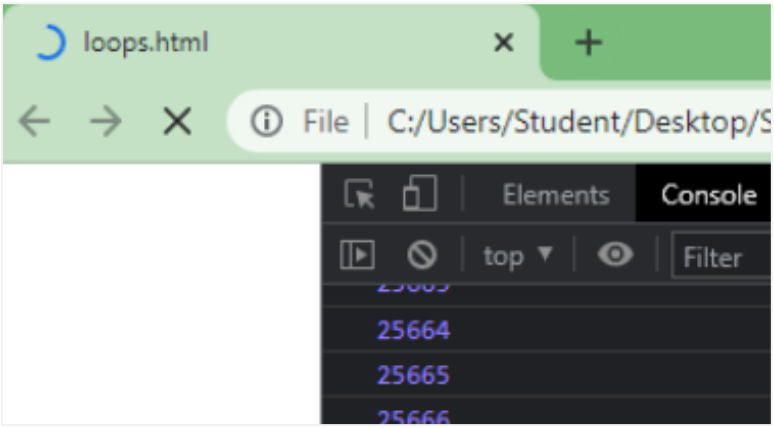


Figure 2 - web browser tab titled "loops.html" displaying an infinite run icon on the top-left corner

for loop in an array

for loop is very useful to loop through each value in an array. When we work with **for** loop in an array, we must keep in mind that zero is the most common initial value because it is the initial index value in an array.

Example 17) Use a for loop to display each item in array **cars[]**

```
let cars =['Mazda','Tesla','Dodge','BMW','Porshe']
for(let i=0; i<cars.length ; i++){
   console.log(`Car ${i+1} = ${cars[i]}`)
}
```

When the program runs, the following output will be displayed in the console:

```
Car 1 = Mazda
Car 2 = Tesla
Car 3 = Dodge
Car 4 = BMW
Car 5 = Porshe
```

There is also a specific statement in a for loop that works with array, which is the **for… of** statement. The **of** statement in a ***for*** loop will loop to each item in the list

Example 18) use a **for ... of** loop to print each item in array

```
let cars = ['Mazda', 'Tesla', 'Dodge','BMW', 'Porshe'];
for(let eachItem of cars){
  console.log(eachItem);
}
```

When the program runs, the following output will be displayed in the console:

```
Mazda
Tesla
Dodge
BMW
Porshe
```

for loop in a string

We can also use **for** loop to go to each of the character in a string:

Example 19) use a **for…of** loop to print each character in a string

```
let myString = "Prof. Wu"
for(let eachLetter of myString)
     console.log(eachLetter);
}
```

When the program runs, the following output will be displayed in the console:

```
P
r
o
f
.

W
u
```

Example 20) Nested statements: create an array of foods. Use a for loop to display a message for each value in the foods array. For example: *"I want some _____."* All foods at even indexes should be displayed in uppercase.

```
let foods =['salad','pizza','tofu','cheese burger'];
for(let item = 0; item<foods.length ; item++){
   if((item+1)%2===0){
        let itemUpper = foods[item].toUpperCase();
        console.log(`I want some ${itemUpper}`)
      }
      else{
        console.log(`I want some ${foods[item]}`)
      }
 }
```

When the program runs, the following output will be displayed in the console:

```
I want some salad
I want some PIZZA
I want some tofu
I want some CHEESE BURGER
```

Nesting for loops

Nesting **for** loops, basically means that **for** each iteration of the outer for loop, it runs one complete inner **for** loop.

Example 21) write a JS code that will print a decrement from 30 to 0, decrement of 10, three times

```
for(let outerCounter = 1 ; outerCounter<= 3 ; outerCounter++){
```

```
   console.log(`OUTER LOOP COUNT \t\t ${outerCounter}`)
   for(let innerCounter = 30 ; innerCounter>=0 ; innerCounter -= 10){
       console.log(`\tInner counter = ${innerCounter}`)
    }
}
```

When the program runs, the following output will be displayed in the console:

```
OUTER LOOP COUNT    1
     Inner counter = 30
     Inner counter = 20
     Inner counter = 10
     Inner counter = 0
OUTER LOOP COUNT    2
     Inner counter = 30
     Inner counter = 20
     Inner counter = 10
     Inner counter = 0
OUTER LOOP COUNT    3
     Inner counter = 30
     Inner counter = 20
     Inner counter = 10
     Inner counter = 0
```

3.5.2. while loop

The **for** loop assumes you know how many times you want to loop to run but sometimes you just want to run the loop until some condition changes. In that case, you can use a **while** loop instead.

The **while** loop allows us to create more advanced functions inside the core block and run the loop as long as these or other external conditions are true.

Syntax

```
while (loop condition) {
     // code block to be executed
}
```

One of the ways to create **while** loop is to have the initial value of the while loop statement declares before the while loop. Once in the while loop, inside the while loop should also have the update of the initial value. Otherwise, if the initial value of the

condition statement of the while loop is not updated, the while loop will run infinitely, which is not recommended!

Syntax

```
let initial_value = value
while (loop condition) {
  // code block to be executed
  update initial_value
}
```

Example 22) use **while** loop to display number from 0 to 4

```
let y = 0; //initial value
while (y<5) {
     console.log(`The current y = ${y}`);
     y++; //update initial value
}
```

When the program runs, the following output will be displayed in the console:

```
The current y = 0
The current y = 1
The current y = 2
The current y = 3
The current y = 4
```

Example 23) Create a JavaScript program using a while loop that asks the user to guess a secret number. If the user enters the wrong number, the program should continue prompting them to try again. The loop stops only when the user correctly guesses the secret number.

```
const SECRET = 8;

let guessNum = parseInt(prompt("Guess a number between 0 and 10"));
while (guessNum !== SECRET){
     guessNum = parseInt(prompt("WRONG! Guess another number: "));
}
```

'break' and 'continue' keyword

break statement

The **break** keyword "jumps out" or terminates the entire loop. You have already seen the **break** statement used in a **switch()** statement. It basically works the same way in loops.

Example 24) Create a JavaScript program that uses a decrementing counter from 20 to 0 (inclusive), but stops the loop when the counter reaches 9.

```
for(let counter = 20; counter>=0 ; counter--){
   if(counter===9){break}
   console.log(counter)
}
```

Example 25) Create a JavaScript program where a while loop continues executing as long as a break condition is not triggered. The loop should run indefinitely (**while(true)**) and only stop when the user enters a character other than **y** or **Y**.

```
let check;
let n;

while (true) {
    n = parseInt(prompt("Enter a number: "));

    let doubled = n * 2;
    console.log(`The double of number ${n} is ${doubled}`);

    check = prompt("Do you want another run? (y/n)").toLowerCase();

    if (check !== 'y' && check !== 'yes') {
        break;
    }
}
```

continue statement

The **continue** statement breaks one iteration (in the loop) if a specified condition occurs, and then continues with the next iteration in the loop.

Example 26) Create a JS code that will **skip** all even numbers from 9 to 0, inclusive.

```
loop_counter = 10
while(loop_counter>0){
    loop_counter--
    if(loop_counter%2===0){
        continue
    }
    console.log(loop_counter)
}
```

Example 27) Create a JavaScript code that will **skip** a number of an increment counter from 0 to 10, inclusive. The skip number is entered by user from a dialog box. The program also validates that the input number is between 0 and 10

```
let skipNum;

while (true) {
  skipNum = parseInt(prompt("Enter a number to skip: "));

  if (!isNaN(skipNum) && skipNum >= 0 && skipNum <= 10) {
    break;
  }

  alert("Invalid input! Enter a number between 0 and 10.");
}

for (let counter = 0; counter <= 10; counter++) {
  if (counter === skipNum) {
    continue;
  }
  console.log(counter);
}
```

3.6. Functions

JavaScript programs can quickly grow into hundreds or even thousands of lines of code. Many parts of this code may repeat, which makes the program longer and harder to manage. To improve structure and reduce repetition, we use **functions** and **objects**.

Functions are small programs inside a larger script. They help organize code into manageable sections and allow us to reuse logic whenever needed. A function contains a block of code that is executed when it is called, and this code can include variable declarations, operations, and conditional statements.

Functions generally serve two purposes: they either perform an action immediately (such as updating content on a webpage), or they produce a value that can be used elsewhere in the program, known as a **return value**. In JavaScript, there are three main types of functions: **named functions**, which run when called by their name; **anonymous functions**, which are often executed in response to events; and **immediately invoked function expressions (IIFE)**, which run as soon as they are defined.

All functions follow a similar structure. They begin with the keyword function, followed by the function name, a pair of parentheses (which may include parameters), and a set of curly braces {} that contain the code to be executed.

To run a named function, we call it by name at a location in the script where we want it to run. That means we can define a function at the top of the script and choose to run it at the bottom of the script. In fact, this is the coding standard for functions.

Technically, it does not matter where a function is in the JavaScript script because the browser will load all the JavaScript's code first and then execute it. But to make it easier for humans to read, we normally place a function before it is called in the script. This provides an appearance of logical structure in our code when you read it from top to bottom.

Finally, functions can return values from where they were called using the return keyword. Everything that is returned is not executed directly, but is captured in a variable or used immediately in another function.

3.6.1 Creating a function: user defined functions

Named functions are useful if we need to call a function many times to pass different values to it or just need to run it several times and it's also useful when functions just get big and just clutter up the overall flow of the script. In that case, we create functions and put them above the main script to be called when needed. Named functions can have parameters and not, or with or without returning value.
Functions are reusable. We can call a function as many times as necessary in our JS program.

Parameters in JS are like a placeholder where you store the value of a variable, known as arguments when the function is called, and pass that value to the function to be used within the function. Basically, parameters are variables that pass values to the function as arguments to obtain the function's result.

A function that has no parameters and does not return a value is a simple function that performs an action without taking any input or producing an output.

Example 28) Define a function that prints `Hello World!` in the console.

```
function msg() {
   console.log('Hello World!')
}
```

Calling the function by typing **`msg(),`** it will print **`Hello World!`** in the console.

Example 29) define a function called **printCount** that prints out the number from 3 to 1, inclusive, every time the function is called.

```
function printCount(){
  for(let num =3; num>0; num -= 1){
    console.log(num)
   }
}
```

When the program runs, the following output will be displayed in the console:
3
2
1

A function that **receives a value through parameters** is one that takes input data and uses it inside the function to perform a specific task.

Example 30) Define a function that takes a parameter called **name** and prints the value of the argument to the console.

```
function greeting(name){
    console.log(`Hello ${name}`);
}
```

If the function is called and passes the argument value of Ana:
```
greeting('Ana')
```

The console will print:
```
Hello Ana
```

If a function is defined with one or more parameters but is called without any arguments, the parameters will default to undefined.

For example, if the **greeting** function is called as:
```
greeting()
```

The console will print:
```
Hello undefined
```

Example 30) Define a function that takes a message as an argument and converts it to uppercase.

```
function upperMessage(message){
  let upper = message.toUpperCase();
  console.log(upper);
}

//call function
upperMessage('Passed Final Exam!')
```

The console will print:
```
PASSED FINAL EXAM
```

The order of arguments is important because each argument is matched to its corresponding parameter based on position.

Example 31) define a function that will take two parameters, *firstName* and *lastName*, and print the first name with the initial letter of the last name.

```
function greet(firstName, lastName){
     console.log(`Hello ${firstName} {lastName.toUpperCase()}`)
}
//call function
greet('Peter', 'Pan')
```

The console will print:
```
Hello Peter PAN
```

Example 32) Define a function that has two parameters: **message** and **numCounts**. The **message** parameter takes a string, and **numCounts** takes a number. Inside the function, print the message value to the console the number of times specified by **numCounts**.

```
function printMsg(message, numCount){
    for(counter=1; counter<=numCount ; counter++){
       console.log(`Counter ${counter} = ${message}`)
    }
}
```

When the program runs, the following output will be displayed in the console:
```
Counter 1 = Hello
Counter 2 = Hello
Counter 3 = Hello
```

Example 33) In some dice games like Craps, rolling two 1s is called “Snake Eyes” and is usually considered an unlucky roll. Define a function called isSnakeEyes that takes two numbers as parameters, representing the values of two dice. If both numbers are 1, the function should print “Snake Eyes!”; otherwise, it should print “Not Snake Eyes!”.

```
function isSnakeEyes(dice1,dice2){
    if (dice1===1 && dice2===1){
        console.log("Snake Eyes!")
    }
    else{console.log("Not Snake Eyes!")}
}
```

```
// calling the function
isSnakeEyes(3,3)
```

The console will print:
```
Not Snake Eyes!
```

```
// calling the function
isSnakeEyes(1,1)
```

The console will print:
```
Snake Eyes!
```

```
// calling the function
isSnakeEyes(1,3)
```

The console will print:
```
Not Snake Eyes!
```

Return values are values that a function returns when it has completed running the code within it. return is exporting the value out of the function. Also, the return of a function stops the execution of a function. Any lines after the return will not run.

Example 34) define a function that returns the sum of two numbers. The two numbers are passed to the function as **x** and **y**

```
function sumxy(x,y){
  return x+y;
  console.log("LINE AFTER RETURN")
}

// call function sumxy
let sum1 = sumxy(2,7)
console.log(sum1);
```

When the program runs, the following output will be displayed in the console:
9

Remember, **return** stops a function, therefore, the line console.log("LINE AFTER RETURN") does not appear in a console.

Example 35) define a function that takes a number as parameter. The function will return the double of the number

```
function doubleNumber(number){
        let dNumber = number*2;
        return dNumber
}

// call function doubleNumber()
let num = 4;
doubleNum=doubleNumber(num);
console.log(`The double of number ${num} is ${doubleNum}`);
```

When the program runs, the following output will be displayed in the console:

```
The double of number 4 is 8
```

Example 36) define a function that adds two numbers and return the sum of them. Save the returned value in a variable **number1** and **number2**

```
let a = 5/9, b = 3, c = 9, d = -5;

function sumNum(num1, num2) {
     let sum = num1 + num2;
     return sum;
}

let number1 = sumNum(10, 30);
let number2 = sumNum(c, d);
```

Example 37) Define a function that checks whether the outdoor temperature is greater than 75. The function should take a single numeric parameter called temperature and return true if the temperature is greater than or equal to 75, and false if it is less than 75.

```
function isShortsWeather(temperature){
    if (temperature>=75){return true;}
    else{return false}
}

// calling the function
isShortsWeather(60)      // false
isShortsWeather(90)      // true
```

Example 38) Define a function that takes a single array as an argument. The function should return the last element in the array. If the array is empty, it should return null.

```
function lastElement(arrayInput){
     if(arrayInput ===undefined){
          return null
     }
     else if(arrayInput.length !==0){
               let lastIndex = arrayInput.length - 1;
               return arrayInput[lastIndex]
          }
          else{
          return null
     }
}

// calling the function
lastElement()                        // null
lastElement([2,-5, "dog",10,-20])  // -20
```

3.6.2. Built-in function and methods. Math object

JavaScript provides number of built-in functions that are global functions. Some of those built-in functions are math object function.

JavaScript math object allows users to perform mathematical tasks on numbers such as to square, round off, or randomly generate a number.

Example 39) apply different Math methods to a PI value

```
let num = Math.PI;

console.log(`PI = ${num}`);

//Math.round() returns the nearest integer
console.log(`PI round =  ${Math.round(num)}`);
// Math.ceil() returns the value rounded up to its nearest
integer
console.log(`PI ceil = ${Math.ceil(num)}`);
//Math.floor() returns the value rounded down to its nearest
integer
console.log(`PI floor = ${ Math.floor(num)}`);
// Math.trunc() returns the integer part of x
console.log(`PI trunc = ${Math.trunc(num)}`);
//Math.pow(base, exponent), returns the value of base to the
power of exponent
console.log(`2^3 =  ${Math.pow(2,3)}`);
//Math.sqrt() returns the square root of a number
console.log(`square root of 81 = ${Math.sqrt(81)}`);
//Math.random() returns a random number between 0 (inclusive),
and 1 (exclusive)
console.log(`Random = ${Math.random()}`);
```

When the program runs, the following output will be displayed in the console:

```
PI = 3.141592653589793
PI round = 3
PI ceil = 4
PI floor = 3
PI trunc = 3
2^3 = 8
Square root of 81 = 9
Random = 0.592085018187762
```

Example 40) create a function that randomly generate a number between 10 and 20 inclusive

```
function pickNum(){
  let pick = 10+Math.round(Math.random()*20);
  return pick;
}
console.log(pickNum()); //should print a number between 10 and 20
```

Example 41) Create a function that randomly pick a color from an array.

```
let colors = ['red','green','blue','yellow','orange'];

function colorPick(lastIndex){
  let pick = Math.round(Math.random()*lastIndex);
  return pick;
}

//call function colorPick()
pickedIndex = colorPick(colors.length - 1)
console.log(`The picked color is = ${colors[pickedIndex]}`)
```

When the program runs, the following output will be displayed in the console:

```
The picked color is = orange
```

If we refresh the browser, another random color is picked:

```
The picked color is = yellow
```

3.6.3. Global and Local Scoping in JavaScript

Scope in JavaScript determines the accessibility of variables. The two types of scope are *local* and *global*. As we already learned, variables hold the data or information that we can access or reassign. The two main type of scooping in JavaScript is **local** and **global** variables.

Local variables: When you use JavaScript, local variables are variables that are defined within functions. They have local scope, which means that they can only be used within the functions that define them.

Global Variables: global variables are variables that are defined outside of functions. These variables have global scope, so they can be used by any function without passing them to the function as parameters.
Function scope: variables declare inside a function is local to the function and not affect the variables outside of the function. If there is a variable declared outside the function and we want to use the same variable inside a function, it will refer the outside variable inside the function.

Example 42) function scope: global variable msg

```
let msg = 'This is a outside message'
function displayMsg(){
        let msg = 'Hello World!'
        console.log(msg)
}

// calling the function
displayMsg()        // Hello World!
```

If we remove the line let **msg = 'Hello World!'** inside the function, the function will use the value of **msg** from the global scope.

```
let msg = 'This is a outside message'
function displayMsg(){
   console.log(msg)
}

// calling the function
displayMsg()        // This is a outside message
```

Example 43) Block scope: global and local variables using var instead of let or const.

```
var fullMoon = true;
// Initialize a global variable
var species = "human";

if (fullMoon) {
   // Initialize a local variable
   var species = "werewolf";
```

```
   console.log(`It is a full moon. Lupin is currently a
${species}.`);
}

console.log(`It is not a full moon. Lupin is currently a
${species}.`);
```

When the program runs, the following output will be displayed in the console:

```
It is a full moon. Lupin is currently a werewolf
It is not a full moon. Lupin is currently a werewolf
```

Avoid using a global scoop as much as possible. To do so, we can convert a global variable into a local variable by placing the function inside a function. This anonymous function runs as soon as the script loads, and it will also keep variables out of the global scope. One way to avoid global scooping, besides changing the variable's name, is to change var to *const* and *let*

```
const fullMoon = true;
// Initialize a global variable
let species = "human";

if (fullMoon) {
// Initialize a block-scoped variable
let species = "werewolf";
    console.log(`It is a full moon. Lupin is currently a
${species}.`);
}

console.log(`It is not a full moon. Lupin is currently a
${species}.`);
```

When the program runs, the following output will be displayed in the console:

```
It is a full moon. Lupin is currently a werewolf
It is not a full moon. Lupin is currently a human
```

Use of const and let

const, constant, variables' are variables whose values cannot be changed later.

Example 44) without constant

```
var msg = "Hello World!";
console.log(msg);
msg = "Good Evening!";
console.log(msg);
```

Changing the previous code with const

```
const msg = "Hello World!";
console.log(msg);
msg = "Good Evening!";
console.log(msg);
```

When the program runs, the following output will be displayed in the console:

```
Hello World!
Uncaught TypeError: Assignment to constant variable.
    at <anonymous>:3:5
```

The ***let*** statement declares a block-scoped local variable, optionally initializing it to a value.

Example 45) without **let**

```
var i = 1;
if (i === 1) {
    var i = 2;
    console.log(`x inside if statement ${i}`);
  }
console.log(`x outside if statement ${i}`);
```

When the program runs, the following output will be displayed in the console:

```
x inside if statement 2
x outside if statement 2
```

Changing the previous code with **let** instead of **var:**

```
let i = 1;
if (i === 1) {
    let i = 2;
    console.log(`x inside if statement ${i}`);
}
console.log(`x outside if statement ${i}`);
```

When the program runs, the following output will be displayed in the console:

```
x inside if statement 2
x outside if statement 1
```

3.6.4. Save function in a variable

We can also save the return value of a function into a variable:

```
const sum = function(num1,num2){
    return num1+num2;
}
```

To call this variable function, we just need to call the variable name and pass the arguments values.

Example 46) define a function that returns the square of a number. Define the function as a function expression, stored in a variable called **square**.

```
let square = function(num){
    return Math.pow(num,2);
}

// calling the function
let n = square(4);
console.log(n)
```

When the program runs, the following output will be displayed in the console:

```
16
```

Anonymous functions

Anonymous function is a function that does not have any name associated with it. Normally we use the *function* keyword before the function name to define a function in JavaScript, however, in anonymous functions in JavaScript, we use only the *function* keyword without the function name. An anonymous function is not accessible after its initial creation; it can only be accessed by a variable it is stored in as a *function as a value*. An anonymous function can also have multiple arguments, but only one expression.

Anonymous functions need to be tied to something, a variable, or an event, or something similar to run. Anonymous functions are used mainly when the function is not called often in different part on the script.

Syntax

```
function() {
    // Function Body
}
```

We may also declare an anonymous function using the **arrow function** technique which is shown below:

```
( () => {
    // Function Body...
} )();
```

Example 47) In this example, we define an anonymous function that prints a message to the console. The function is then stored in the *greet* variable. We can call the function by invoking *greet().*

```
let greet = function (n) {
        console.log(`Welcome to  ${n}`);
};

//calling the function
greet("GeeksforGeeks!");
```

When the program runs, the following output will be displayed in the console:

```
Welcome to GeeksforGeeks!
```

Arrow functions

ES6 introduced a new and shorter way of declaring an anonymous function, which is known as **Arrow Functions.** In an Arrow function, everything remains the same, except here we don't need the *function* keyword also. Here, we define the function by a single parenthesis and then '=>' followed by the function body.

Example 48) in this example, we are going to rewrite the Anonymous in the previous example into an arrow function

```
let greet = (n) => console.log(`Welcome to ${n}`);

//calling the function
greet("GeeksforGeeks!");
```

(argha_c14, 2023)

3.6.5. Higher order functions, functions calling functions

Higher-order functions are functions that operate on other functions. This occurs when a function is passed as an argument to another function or when a function returns another function as its result.

Example 49) define a function that will run twice the function **rolldie()**

```
function callTwice(rep){
   rep();
   rep();
}
function rolldie(){
   const roll = Math.floor(Math.random()*6+1)
   console.log(roll)
}
//calling the function
callTwice(rolldie)
```

When the program runs, the following output will be displayed in the console:
4
2

Returning functions

A function can also be nested inside of another function.

Example 50) A factory function is a function that returns another function. The outer function first receives and processes an argument, and then returns an inner function. The returned function can be stored in a variable and called later when needed.

```
function makeBetweenFunc(min,max){
  return function (num){
  return num>=min && num<=max;
  }
}
```

This function returns another function that checks whether a given number falls within a specified range.

To use it, we can store the returned function in a variable:

```
let child = makeBetweenFunc(2, 6);

console.log(child(5))    // true
```

Now, the variable **child** acts as a function. When we call it with a value such as **child(5)**, it returns **true** because 5 is between 2 and 6.

However, if we call **child(7),** it returns **false** because 7 is outside the defined range.

```
console.log(child(7))    // false
```

Default parameters

Default values in a parameter is set when the argument is not passed as required. In case that the passed argument is not properly passed, the argument will set to the default value in the parameter.

Example 51) Create a function that prompts the user to enter the number of faces on a die. If no value is provided, the function should use a default parameter value instead.

```
function rollDie(numSides){
   return Math.floor(Math.random()*numSides)+1
}
```

Changing the parameter in the **rollDie** with a default value of **6**

```
function rollDie(numSides=6){
      return Math.floor(Math.random()*numSides)+1
}
```

Since parameters order matters, it is recommended to have a default parameter as last in the order of parameters.

Example 52) setting different default parameters

```
function greeting(msg,name){
      console.log(`${msg}, ${name}!`)
}
```

Setting a default value to parameter **msg**

```
function greeting(msg='Hey there',name){
    console.log(`${msg}, ${name}!`)
}
```

We can also change the order of the default parameters:

```
function greeting(name,msg='Hey there'){
    console.log(`${msg}, ${name}!`)
}
```

We can also have multiple default parameters:

```
function greeting(name,msg='Hey there', punc='!!'){
    console.log(`${msg}, ${name} ${punc}`)
}
```

3.6.6. Spread ...

spread syntax(**...**) allows an iterable such as an array to be expanded in places where zero or more arguments (for function calls) or elements (for array literals) are expected, or an object expression to be expanded in places where zero or more key-value pairs (for object literals) are expected.

Example 53) use spread syntax to find the maximum number of an array

```
nums =[-34,23,89,-10,0,56,-92]
let num1 = Math.max(nums)
// since Math.max() takes since values in the parenthesis to
compare which one of them is the greatest.
```

We can use the spread syntax to pass the **nums** array as individual values into the **Math.max()** method.

```
nums =[-34,23,89,-10,0,56,-92]
let num1 = Math.max(...nums)
```

Spread allows array to have an independent copy on an array

Example 54) use spread syntax to copy two arrays into one array

```
let colors =['red','blue','yellow']
let animals =['cat', 'dog']

let comArray = [...colors]
```

We can also combine arrays using the spread syntax:

```
let comArray = [...colors, ...animals];
console.log(comArray)
```

When the program runs, the following output will be displayed in the console:

```
['red', 'blue', 'yellow', 'cat', 'dog']
```

Example 55) with **let**

```
for (let i = 0; i < 10; i++) {
    console.log(i);
}
console.log(`The last value of i is ${i}`);
```

3.7. Try and catch

When an **error** occurs, JavaScript will stop and generate an error message. To avoid the program to stop, can have the program to generate an error message and continue with the program instead of stopping the program. The technical term for this is JavaScript **throws an exception**. JavaScript creates an **Error object** with two properties**: name and message.**

The exceptions in JavaScript are:

- The **try-catch-finally** statements combo handles errors without stopping JavaScript.
- The **try** statement defines the code block to run (to try).
- The **catch** statement defines a code block to handle any error.
- The **finally** statement defines a code block to run regardless of the result.
- The **throw** statement defines a custom error.
- Both **catch** and **finally** are optional, but you must use one of them.

Syntax

```
try {
     Code block to run
}
catch(err) {
     Code block to handle errors
}
finally {
     Code block to be executed regardless of the try result
}
```

Example 56) Apply exception handling to display an error in the console if the input collected from the prompt is not a string.

```
try{
     msg = prompt('Enter a message')
     console.log(msg.toUpperCase().repeat(3))
}catch{
     console.log('ERROR! Please pass a string next time!')
}
console.log('AFTER!')
```

Implementing exception handling inside a function and displaying the type of error in the catch block would look like this:

```
function yell(msg){
     try{
        msg = prompt('Enter a message')
        console.log(msg.toUpperCase().repeat(3))
     }catch(e){
        console.log(`${e} Please pass a string next time!`)
     }
}
```

In JavaScript, you can throw exceptions using the throw statement. This is useful for handling errors in your code. Here's how it works:

```
function checkNumber(value) {
  if (typeof value !== "number") {
    throw new Error("Invalid input: value must be a number");
  }

  console.log("Valid number:", value);
}

try {
  checkNumber("hello");
} catch (error) {
  console.log("Error caught:");
  console.log(error.message);
}
```

When the program runs, the following output will be displayed in the console:

```
Error caught:
Invalid input: value must be a number
```

What happens:

- **checkNumber("hello")** is called
- Since "hello" is not a number, the function **throws an error**
- The catch block catches it and prints the error message

Throwing Different Types of Exceptions

1. Throwing a String
   ```
   throw "Something went wrong!";
   ```
2. Throwing a Number
   ```
   throw 404;
   ```

3. Throwing an Object
```
throw { message: "Invalid input", code: 400 };
```
4. Throwing an Error Object (Recommended)
```
throw new Error("An error occurred!");
```

Using throw with try...catch

To handle thrown exceptions, use a try...catch block:

```
try {
    throw new Error("Something went wrong!");
} catch (error) {
    console.log(`Caught an error: ${error.message}`);
}
```

Example 57) Define a JavaScript function named '**divideNumbers**' that takes two parameters, '**a**' and '**b**'. The function should attempt to divide '**a**' and '**b**' and return the result. If the division is undefined:

- return a message that says 'Error performing the division' if b is equal to zero.
- Return a message that says if 'Must enter a number' if 'a' or 'b' is not a number.
- The program must use try-catch inside the function.

```
function divide(a, b) {
    try {
        if (b === 0) {
            throw new Error("Division by zero is not allowed.");
        }
        if (isNaN(a) || isNaN(b)){
            throw new Error("Must enter a number.");
        }
        return a / b;
    } catch (error) {
        console.error(`An error occurred: ${error.message}`);
        return 0;
    } finally {
        console.log("Execution completed.");
    }
}

// calling the function
divide(10, 2);
// Execution completed. 5

divide(10, 0);
```

```
// An error occurred: Division by zero is not allowed.
// Execution completed.

divide("a", 2);
// An error occurred: Must enter a number.
// Execution completed.
```

Example 58) create an error handling to throw an error if the index is out of range in a list. The program will have the following requirement:

- A function named 'getArrayElement' checks if the index is within valid bounds.
- If the index is out of range, it throws a RangeError.
- The try block attempts to access array elements.
- The catch block handles RangeError and logs an appropriate message.
- The finally block ensures the completion message is logged regardless of success or failure.

```
function getArrayElement(arr, index) {
    if (index < 0 || index >= arr.length) {
        throw new RangeError("Index out of bounds.");
    }
    return arr[index];
}

try {
    let numbers = [10, 20, 30, 40, 50];
    console.log(getArrayElement(numbers, 2)); // Outputs: 30
    console.log(getArrayElement(numbers, 10)); // Throws an exception
} catch (error) {
    if (error instanceof RangeError) {
        console.error("RangeError:", error.message);
    } else {
        console.error("An unexpected error occurred:", error);
    }
} finally {
    console.log("Array access operation completed.");
}
```

3.8. JavaScript Objects

3.8.1. object definitions

In real life, a car is an **object**. A car has **properties** like weight and color, and **methods** like start and stop:

Object:

- `car`

Properties:

- `car.name = Fiat`
- `car.model = 500`
- `car.weight = 850kg`
- `car.color = white`

methods

- `car.start()`
- `car.drive()`
- `car.brake()`
- `car.stop()`

All cars have the same **properties**, but the property **values** differ from car to car.
All cars have the same **methods**, but the methods are performed **at different times**.
You have already learned that JavaScript variables are containers for data values.
Objects are variables too. But objects can contain many values.

This code assigns **many values** (Fiat, 500, white) to a **variable** named car. The values are written as **name:value** pairs (name and value separated by a colon).

You define (and create) a JavaScript object with an object literal. It is common practice to declare objects with the const keyword.

Syntax

```
const object_name = {property_name:value}
```

Example 59) create a **car** object with properties **type, model**, and **color**

```
// Create an object:
const car = {type:"Fiat", model:"500", color:"white"};
console.log(car.type)
```

Spaces and line breaks are not important. An object definition can span multiple lines:

```
const car = {
type:"Fiat",
model: "500",
color:"white"
};
```

Accessing Object Properties
You can access object properties in two ways:
`objectName.propertyName`

or

`objectName["propertyName"]`

3.8.2. What is “this”?

In JavaScript, the **this** keyword refers to an object, which object depends on how this is being invoked (used or called).

The **this** keyword refers to different objects depending on how it is used:

- In an object method, this refers to the **object**.
- Alone, this refers to the **global object.**
- In a function, this refers to the **global object.**
- In a function, in strict mode, this is undefined.
- In an event, **this** refers to the **element** that received the event.
- Methods like call(), apply(), and bind() can refer this to **any object**

Note
this is not a variable. It is a keyword. You cannot change the value of **this**.

The “ this” keyword
In a function definition, this refers to the "owner" of the function. In the example above, this is the **person object** that "owns" the **fullName** function.

In other words, **this.firstName** means the **firstName** property of **this object**.

Example 60) create an object method using *this* keyword

```
const person = {
  firstname: "john",
  lastname: "doe",
  id: 5566,
  fullname: function() {
    return this.firstname + " " + this.lastname;
  }
};

console.log(person.fullname())
```

(w3school.com, 2023)

3.8.4. Object constructor using functions

Object Constructors are templates for creating objects that we define once, and then we can use those templates repeatedly.

To make a constructor for any object we start with function, then we give our function a name and here we capitalize the first letter to signify **this** is an object.

Example 61) create a constructor course with properties title, instruction, code, session, and number of students:

```
function Course(title, instructor, code, session, students){
     this.title = title;
     this.instructor = instructor;
     this.code = code;
     this.session= session;
     this.number_students = students;
};
// call for object function
let course1 = new Course("Creating Smartphone Apps", "Prof.
Wu", "ET570","M4",20);
let course2 = new Course("Computer Applications", "Prof.
Novak","TECH100","C3",26);
// accessing object function
console.log(course1.instructor)
```

3.8.5. Methods

A method is a function that is placed as a property in an object. Every method is a function but that every function is a method.

Example 62)

```
const myMath = {
     square:function (num){return num*num},
     double:function(num){return num+num}
}
```

Newest version of JS do not require to write the function keyword in a method:

```
const myMath = {
   square (num){return num*num},
   double (num){return num+num}
}
```

Example 63) define an object called square, which will hold methods that have to do with geometry of squares. It should contain two methods: area and perimeter:

- area should accept the length of a side and then return the side squared.
- Perimeter should accept the length of a side and return that side multiplied by 4

```
const square = {
    area (side){return side*side},
    perimeter(side){return side*4}
}
```

"this" in method

Use the keyword ***this*** to access other properties on the same object.

Example 64) create an object ***cat*** with properties: **name, color, breed,** and method **meow()**

```
const cat={
          name: 'Mickey',
          color: 'white with black spots!',
          breed: 'unknown',
          meow(){ console.log('MEOW MEOW MEOW')
          }
 }
```

```
// If you call the method meow()
cat.meow();
```

When the program runs, the following output will be displayed in the console:
```
MEOW MEOW MEOW
```

We can use the accessor ***this*** in method **meow()** to print the property ***name*** within the same object:

```
const cat={
          name: 'Mickey',
          color: 'white with black spots!',
          breed: 'unknown',
          meow(){ console.log(this.name)
          }
}

// If you call the method meow()
cat.meow();
```

When the program runs, the following output will be displayed in the console:
```
Mickey
```

Example 65) Use of accessor ***this*** in an object
Define an object called **hen**. It should have three properties:

- **name** should be set to Helen
- **eggCount** should be set to 0
- **layAnEgg** should be a method which increments the value of eggCount by 1 and return the string EGG

When the class is executed, calling the **layAnEgg** method will print "EGG" to the console and increase the value of the **eggCount** property by **1**. As a result, if you access the **eggCount** property afterward, it will display 1, as shown below:

```
const hen = {
    name: 'Helen',
    eggCount: 0,
    layAnEgg(){this.eggCount++; return 'EGG'}
}
```

Bibliography

- Duckett, J. (2016). *HTML and CSS Design and build websites.* Indianapolis: John Wiley and Sons Inc.
- Duckett, J. (2016). *JavaScript and JQuery: interactive front-end developer.* Indianapolis: John Wiley and Sons Inc.
- *HTML, CSS, JavaScript, JQuery, and Bootstrap*. (2017, June). Retrieved from w3schools: www.w3schools.com
- Some material compiled from www.lynda.com (May 2018), www.w3schools.com (2020), www.coursera.org (2020)
- argha_c14. (2023). *JavaScript Anonymous Functions*. Obtenido de Geeks for Geeks: https://www.geeksforgeeks.org/javascript-anonymous-functions/
- w3school.com. (2023). *JavaScript Objects*. Obtenido de w3school.com: https://www.w3schools.com/js/js_objects.asp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures
https://developer.mozilla.org/en-US/docs/Glossary/IIFE

Reference

Table 1: Array methods and their corresponding descriptions	
Array Method	**Description**
`concat()`	Joins two or more arrays, and returns a copy of the joined arrays
`copyWithin()`	Copies array elements within the array, to and from specified positions
`entries()`	Returns a key/value pair Array Iteration Object
`every()`	Checks if every element in an array pass a test
`fill()`	Fill the elements in an array with a static value
`filter()`	Creates a new array with every element in an array that pass a test
`find()`	Returns the value of the first element in an array that pass a test
`findIndex()`	Returns the index of the first element in an array that pass a test
`forEach()`	Calls a function for each array element
`from()`	Creates an array from an object
`includes()`	Check if an array contains the specified element
`indexOf()`	Search the array for an element and returns its position
`isArray()`	Checks whether an object is an array
`join()`	Joins all elements of an array into a string
`keys()`	Returns a Array Iteration Object, containing the keys of the original array
`lastIndexOf()`	Search the array for an element, starting at the end, and returns its position
`map()`	Creates a new array with the result of calling a function for each array element
`pop()`	Removes the last element of an array, and returns that element
`push()`	Adds new elements to the end of an array, and returns the new length
`reduce()`	Reduce the values of an array to a single value (going left-to-right)

`reduceRight()`	Reduce the values of an array to a single value (going right-to-left)
`reverse()`	Reverses the order of the elements in an array
`shift()`	Removes the first element of an array, and returns that element
`slice()`	Selects a part of an array, and returns the new array
`some()`	Checks if any of the elements in an array pass a test
`sort()`	Sorts the elements of an array
`splice()`	Adds/Removes elements from an array
`toString()`	Converts an array to a string, and returns the result
`unshift()`	Adds new elements to the beginning of an array, and returns the new length
`valueOf()`	Returns the primitive value of an array

© 2026. This OER resource is licensed under a Creative Commons license (BY-NC-SA). It may be freely shared and redistributed for non-commercial purposes, provided proper attribution is given and any derivatives are shared under the same license.

Unauthorized copying, reproduction, or distribution of this material, in whole or in part, without permission is not allowed.